

คู่มือโปรเจกต์ · V1.0

Ai102

เครื่องมือสร้างพรีเซนเทชันด้วย AI แบบรันในเครื่อง (local-first) — ใช้ Ollama เท่านั้น ไม่ต้องพึ่ง Cloud LLM และไม่มีระบบล็อกอินกัน

Repository	github.com/Suppakris/Ai102
Live deployment	ai102-psi.vercel.app
พัฒนาต่อจาก	ALLWEONE's presentation-ai (fork ตัดฟีเจอร์ลงเพื่อใช้ทำงานส่งอาจารย์)
License	MIT
Framework	Next.js 16 · React 19 · TypeScript

สารบัญ

- 1. ภาพรวมโปรเจกต์
- 2. เอกสารกับโค้ดจริง — จุดที่ไม่ตรงกัน
- 3. สิ่งต่างจาก Upstream
- 4. ฟิเจอร์
- 5. Tech Stack
- 6. เริ่มต้นใช้งาน
- 7. Environment Variables
- 8. การตั้งค่าฐานข้อมูล
- 9. การรันบน Docker
- 10. Team Workflow
- 11. วิธีใช้งาน
- 12. โครงสร้างโปรเจกต์
- 13. ปัญหาที่รู้อยู่แล้ว
- 14. การมีส่วนร่วม (Contributing)
- 15. License

1. ภาพรวมโปรเจกต์

Ai102 คือเครื่องมือสร้างพรีเซนเทชันด้วย AI ที่รันแบบ local-first เป็น fork ที่ปรับแต่งจากโปรเจกต์โอเพนซอร์ส `presentation-ai` ของ ALLWEONE โดยตัดทอนให้เหลือใช้เฉพาะ Ollama เพื่อใช้ทำงานส่งในวิชาเรียน จุดเด่นคือการ generate ข้อความทั้งหมดรันบนโมเดล Ollama ในเครื่อง ไม่ต้องสมัคร Cloud LLM และไม่มีหน้าล็อกอินมาก่อนใช้งาน

แอปรับหัวข้อจากผู้ใช้ สร้าง outline ที่แก้ไขได้ จากนั้นสร้างเป็นสไลด์เต็มรูปแบบพร้อมรูปภาพที่ AI สร้างให้ รีมที่แก้ไขได้ rich text กราฟ และ export เป็น PowerPoint ได้ — ทำได้ทั้งผ่าน chat agent ในตัว editor และการแก้ไขด้วยมือโดยตรง

2. เอกสารกับโค้ดจริง — จุดที่ไม่ตรงกัน

โค้ดของ repo นี้เปลี่ยนเร็วมาก และเอกสารที่เขียนไว้ (`README.md` , `CONTRIBUTING.md` , comment ใน `docker-compose.yml`) ไม่ได้อัปเดตตามกัน ก่อนจะเชื่อว่าข้อความไหนในเอกสารเป็นฟีเจอร์ที่ใช้งานได้จริง คู่มือฉบับนี้ได้ตรวจสอบไขว้กับซอร์สโค้ดจริงใน `src/` , `prisma/schema.prisma` , `src/env.js` และ `.env.example` โดยตรง พบจุดที่ไม่ตรงกัน 3 จุด ดังนี้

1. เส้นทาง OpenRouter / `LLM_PROVIDER` ใน Docker เป็นโค้ดที่ตายแล้ว

`docker-compose.yml` ยังเช็ตตัวแปร `LLM_PROVIDER` , `OPENROUTER_API_KEY` , `OPENROUTER_BASE_URL` และ `OPENROUTER_DEFAULT_MODEL` ไว้ และในส่วน Docker ของ README ก็อธิบายว่าตั้ง `LLM_PROVIDER=openrouter` เพื่อข้าม Ollama ไปเลยได้

ความจริง: ตัวแปรเหล่านี้ไม่ถูกอ่านที่ไหนใน `src/` เลย ไม่ว่าจะใน `src/env.js` หรือ `model-picker.ts` ก็ไม่มี ในทางปฏิบัติแอปนี้ใช้ Ollama อย่างเดียว ไม่ว่าจะตั้งค่าตัวแปรพวกนี้เป็นอะไรก็ตาม ให้อธิบายเส้นทาง OpenRouter นี้ใช้งานไม่ได้ จนกว่าจะมีใครไปเขียนโค้ดรองรับจริง ๆ

2. Comment ของ `OPENAI_API_KEY` ใน `.env.example` เป็นข้อมูลเก่า

Comment เหนือ `OPENAI_API_KEY` ใน `.env.example` เขียนว่า "ไม่จำเป็นต้องตั้ง เพราะผู้ใช้สามารถใส่คีย์ของตัวเองได้ที่หน้า Settings → 'AI Provider & API Key' ในแอป (แผน Groq ฟรีใช้ได้)"

ความจริง: ไม่มีหน้า Settings แบบนั้น ไม่มีระบบ BYOK และไม่มีการเชื่อมต่อ Groq อยู่ในโค้ดปัจจุบันเลย (ตรวจสอบแล้วด้วยการค้นทุกไฟล์ `.ts` / `.tsx`) นี่คือการที่หลงเหลือมาจากก่อนที่จะ fork ให้เหลือแค่ Ollama ส่วนคำกล่าวอ้างอีกจุดใน README ที่บอกว่าการ generate ข้อความใช้ Ollama อย่างเดียว และตัด cloud provider ออกหมดแล้ว คือคำกล่าวที่ตรงกับโค้ดจริง

3. "Slide audit agent" ยังอยู่ระหว่างพัฒนา ไม่ใช่ฟีเจอร์ที่ใช้งานได้แล้ว

ส่วน Team Workflow ของ README บอกให้ทีม Prompt Testing ใช้ "slide audit agent" ในการตัดสินคุณภาพผลลัพธ์

ความจริง: ไม่มี API route นี้อยู่ได้ `src/app/api` เลย สิ่งที่มีจริงคือโมเดล `SlideAudit` ใหม่ใน `prisma/schema.prisma` (เพิ่มเข้ามาประมาณ 13 ชั่วโมงก่อนคู่มือฉบับนี้จะถูกเขียน ตาม commit history) ซึ่งกำหนดฟิลด์อย่าง `score` , `pass` , `claims` และ `revisionNotes` ไว้ — แต่ยังไม่มีย agent, tool หรือ route ใด ๆ ที่เติมข้อมูลลงไปในตารางนี้ พุดง่าย ๆ คือ โครงสร้างฐานข้อมูลมาแล้ว แต่ฟีเจอร์ที่จะใช้มันยังไม่มา

ส่วนที่เหลือในคู่มือนี้ — Next.js 16.2.1 / React 19.2.4, การ stub demo user ใน `auth.ts` , ตระก่ Ollama-only ใน `model-picker.ts` , สคริปต์ pnpm และ path ไฟล์สำหรับ generate รูปภาพ — ได้ตรวจสอบกับซอร์สโค้ดโดยตรงแล้วว่าตรงกับเอกสารที่เขียนไว้

3. สิ่งต่างจาก Upstream

Fork นี้ปิดทุกอย่างที่ต้องใช้บัญชีแบบเสียเงินหรือระบบล็อกอิน เพื่อให้รันเป็นงานส่งที่ครบในตัวเองได้:

- **ปิด Auth ทั้งหมด** — `src/server/auth.ts` ถูก stub ให้คืนค่า demo admin user คงที่เสมอ ไม่ต้องใช้ Google OAuth ไม่ต้องตั้งตัวแปร `NEXTAUTH_*` เพราะ role ของ demo user เป็น `ADMIN` ตลอด ฟีเจอร์ "admin-only" ทุกอย่างเลยเปิดใช้งานได้กับทุกคนใน build นี้โดยปริยาย ถ้าจะเอาระบบล็อกอินจริงกลับมา ต้อง revert ไฟล์นี้ แล้วตั้งตัวแปร Google/NextAuth ใหม่
- **Text generation ใช้ Ollama อย่างเดียว** — cloud text provider ทั้งหมด (OpenAI, LM Studio, OpenRouter, Groq/BYOK) ถูกเอาออกจาก `src/lib/model-picker.ts` แล้ว ทุก request จะไปจบที่โมเดลที่เสิร์ฟผ่าน endpoint แบบ OpenAI-compatible ของ `OLLAMA_BASE_URL` ค่า provider เก่า (`openai` , `lmstudio`) ที่อาจหลงเหลือจาก client state เก่าจะถูกดักและ redirect ไปใช้โมเดล Ollama เริ่มต้นแทนเฉียบ ๆ โดยไม่ error
- **การ generate รูปภาพยังใช้ cloud provider อยู่** — FAL (Flux models) เป็น path หลักและ default (`src/app/_actions/presentation/generate-slide-image.ts`) ส่วน Together AI เป็น path รอง/legacy ที่ยังมีอยู่เช่นกัน (`src/app/_actions/image/generate.ts`)

4. ฟังก์ชัน

ฟังก์ชันหลัก

- สร้าง outline ด้วย AI ก่อน แล้วค่อยสร้างสไลด์เต็มรูปแบบ ระบบโมเดล Ollama ในเครื่อง
- แก้ไข outline ได้ก่อนจะสรุปเป็นสไลด์จริง
- สร้างสไลด์แบบ real-time
- Auto-save
- Chat agent ในตัว editor (`presentation_agent` สร้างบน LangGraph พร้อม memory ที่เก็บใน Postgres) ที่แก้ไขเดคที่มีอยู่ได้: เปลี่ยน layout/background ของสไลด์, แทนที่หรือ regenerate รูปภาพ, สร้าง/ลบ/regenerate สไลด์, สลับหรือสร้างธีมใหม่ และรัน web search ได้ (ต้องมี `TAVILY_API_KEY`)

ดีไซน์และการปรับแต่ง

- มีธีมสำเร็จรูปให้เลือกหลายแบบ (ดูที่ `src/lib/presentation/themes.ts`)
- สร้างธีมเองได้
- Import ธีมจากไฟล์ PPTX ได้

เครื่องมือสำหรับฟรีเซนต์

- ฟรีเซนต์ได้จากในแอปโดยตรง
- สร้างลิงก์แชร์แบบสาธารณะ
- Export เป็นไฟล์ PowerPoint (`.pptx`)
- ใส่กราฟ, infographic, ฟังสื่อต่าง ๆ ได้
- แก้ไข rich text ผ่าน Plate Editor
- คอมเมนต์/พูดคุยแทรกในเนื้อหาสไลด์ได้

รูปภาพ

- สร้างรูปด้วย AI ผ่าน FAL (โมเดลตระกูล Flux — default คือ Flux 2 Flash; โมเดล Flux/GPT-image อื่น ๆ ถูกบล็อกไว้ให้ admin เท่านั้น แต่ก็ไม่มีผลอะไรเพราะทุกคนเป็น admin ใน build นี้)
- สร้างรูปสำรองผ่าน Together AI (ตระกูล FLUX.1)
- รูปสื่อดึงจาก Unsplash, Pixabay และ Giphy
- ค้นหารูปผ่าน Google Custom Search (`src/app/_actions/apps/image-studio/google.ts`)

5. Tech Stack

หมวดหมู่	เทคโนโลยี
Framework	Next.js 16, React 19, TypeScript
Styling	Tailwind CSS v4
ฐานข้อมูล	Supabase ร่วมกับ Prisma ORM
Text Generation	Ollama (รันในเครื่อง, endpoint แบบ OpenAI-compatible) ผ่าน LangChain + LangGraph agent
Image Generation	FAL (โมเดล Flux, path คลัง), Together AI (path สำรอง)
UI Components	Radix UI
Text Editor	Plate Editor (<code>platejs</code>)
File Uploads	UploadThing
Lint / Format	Biome (ตัวหลัก — ดูสรุปได้ <code>lint</code> / <code>check</code>)

6. เริ่มต้นใช้งาน

สิ่งที่ต้องมีก่อน

- Next.js 16
- pnpm (repo pin เวอร์ชันไว้ที่ `pnpm@11.1.3` ; ใช้ `pnpm-lock.yaml` / `pnpm-workspace.yaml`)
- ฐานข้อมูล PostgreSQL (ทดสอบแล้วว่าใช้ได้ทั้ง Supabase และ Neon)
- ติดตั้งและรัน [Ollama](#) ในเครื่อง พร้อม pull โมเดลไว้อย่างน้อย 1 ตัว (เช่น `ollama pull llama3.2:3b`)
- API key ของ provider เสริม (optional) ตามฟีเจอร์ที่ต้องการใช้ (ดูหัวข้อ 7)

การติดตั้ง

1. Clone repository

```
git clone https://github.com/Suppakris/Ai102.git
cd Ai102
```

2. ติดตั้ง dependencies

```
pnpm install
```

สคริปต์ `postinstall` จะรัน `prisma generate` ให้อัตโนมัติ

3. ตั้งค่า environment variables

```
cp .env.example .env
```

กรอกอย่างน้อย `DATABASE_URL` ที่เหลือดูได้ในหัวข้อ 7

4. ตั้งค่าฐานข้อมูล — ดูหัวข้อ 8

5. เปิด Ollama แล้ว pull โมเดล (ถ้ายังไม่ได้ทำ)

```
ollama pull llama3.2:3b
```

6. รัน dev server

```
pnpm dev
```

แอปจะรันที่ `http://localhost:3000` และ redirect ไป `/presentation` ทันที — ไม่มีหน้า marketing แยกต่างหาก และไม่มีหน้าล็อกอิน (ดูหัวข้อ 3)

7. Environment Variables

Copy `.env.example` ไปเป็น `.env` แล้วกรอกค่าที่จำเป็น `.env.example` คือแหล่งข้อมูลที่ถูกต้องการที่สุด ตัวแปรที่สำคัญมีดังนี้

ตัวแปร	จำเป็นไหม?	ใช้ทำอะไร
<code>DATABASE_URL</code>	จำเป็น	Connection string ของ Postgres ใช้ทั้งตอนรันแอปและตอน migrate สำหรับ Supabase ให้ใช้ Transaction pooler (พอร์ต 6543) — แอปนี้รันบน Vercel serverless functions ถ้าใช้ Session/Direct connection การเชื่อมต่อจะถูกถือครองไว้ตลอดอายุ client แทนที่จะถูกปล่อยคืนทุกครั้งหลัง query ทำให้ max-clients ของ pooler หมดเร็วมาก
<code>FAL_API_KEY</code>	Optional	Provider หลักสำหรับสร้างรูปด้วย AI (Fal/Flux) ถ้าไม่ตั้งค่า การสร้างรูปจะ fail แบบ graceful พร้อม error "not configured"
<code>OLLAMA_BASE_URL</code>	Optional	ซีปที่ Ollama instance ระบุ/ผ่าน tunnel (เช่นผ่าน ngrok) แทนที่จะเป็น localhost
<code>OLLAMA_DEFAULT_MODEL</code>	Optional	Override โมเดลเริ่มต้น (ปกติคือ <code>llama3.2:3b</code>)
<code>OLLAMA_NUM_CTX</code>	Optional	ขนาด context window เป็น token (default: 8192) ค่า default ของ Ollama เอง (มักอยู่ที่ 2048) เล็กกว่า system prompt ตอน generate ของแอปนี้ ทำให้โมเดลเห็นคำสั่งที่ถูกตัดตอน และสร้างเดคท์ที่พังออกมา ให้ลดค่านี้เฉพาะตอนที่ request generate เริ่ม timeout บนเครื่องที่ช้าหรือใช้ CPU ล้วน
<code>OLLAMA_MAX_OUTPUT_TOKENS</code>	Optional	จำนวน token สูงสุดต่อ 1 request generate ไม่ตั้งค่าไว้โดย default trade-off เหมือนกับ <code>OLLAMA_NUM_CTX</code>
<code>TOGETHER_AI_API_KEY</code>	Optional	Path สำรองสำหรับสร้างรูป (Together AI โมเดล FLUX)
<code>UPLOADTHING_TOKEN</code>	Optional	ที่เก็บรูปภาพที่ AI สร้างขึ้น
<code>UNSPLASH_ACCESS_KEY</code>	Optional	ค้นหารูปสต็อก
<code>GOOGLE_CUSTOM_SEARCH_API_KEY</code> + <code>SEARCH_ENGINE_CX</code>	Optional	ค้นหารูปผ่าน Google Custom Search
<code>TAVILY_API_KEY</code>	Optional	เครื่องมือ web search สำหรับตอน generate outline และ chat agent ใน editor
<code>OPENAI_API_KEY</code>	Optional, ไม่ได้ถูกใช้จริง	ประกาศไว้ใน <code>src/env.js</code> แต่ไม่มีโค้ดส่วนไหนอ่านค่านี้เลย comment ใน <code>.env.example</code> ที่พูดถึงหน้า Settings แบบ BYOK "AI Provider & API Key" เป็นข้อมูลเก่า ไม่มีหน้านั้นอยู่จริง เ็นว่างไว้ได้เลย เอกสารเก่า — ดูหัวข้อ 2

ทุก integration ที่เป็น optional จะ degrade แบบ graceful เมื่อไม่ตั้งค่า — ฟีเจอร์ที่ต้องใช้มันจะแค่ error เฉย ๆ ไม่ crash ตัวแปรเกี่ยวกับ auth (`GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET` / `NEXTAUTH_SECRET` / `NEXTAUTH_URL`) ถูกคอมเม้นต์ไว้และไม่จำเป็นต้องใช้ เพราะระบบล็อกอินถูก stub ไว้แล้ว

8. การตั้งค่าฐานข้อมูล

Schema จัดการด้วย **SQL migrations** (`prisma/migrations/`) หลังจากตั้งค่า `DATABASE_URL` แล้ว:

```
pnpm db:migrate:deploy # prisma migrate deploy — apply migration ที่ค้างอยู่ทั้งหมด
pnpm db:studio         # เสริม: เปิดดูฐานข้อมูลผ่าน Prisma Studio
```

เมื่อแก้ไข `prisma/schema.prisma` ให้สร้าง migration ใหม่ (จะ apply ให้กับ dev database ด้วยในตัว):

```
pnpm db:migrate:dev # prisma migrate dev — สร้าง SQL migration ใหม่จากความต่างของ schema
```

ต้อง commit ไฟลเดอร์ที่ generate ออกมาใน `prisma/migrations/` ด้วย — ไฟล์ SQL นั้น*คือการเปลี่ยนแปลง schema* เอง `pnpm db:push` ยังมีอยู่สำหรับทดลองแบบทิ้งขว้าง แต่มันข้าม migration history ไปเลย อย่าใช้กับฐานข้อมูลที่แชร์กับคนอื่น

ถ้าจะย้ายฐานข้อมูลที่ใช้ `db push` อยู่แล้ว: ถ้าฐานข้อมูลมี schema อยู่แล้ว (สร้างด้วย `db:push` ตั้งแต่ก่อนจะมีระบบ migration) ให้ baseline ครั้งเดียวแทนที่จะรัน migration แรกซ้ำ:
`pnpm exec prisma migrate resolve --applied 0_init`

ไม่ต้องมีขั้นตอน seed แยกต่างหากเพื่อบูตแอป — demo user ที่ stub ไว้จะถูก upsert ให้อัตโนมัติตั้งแต่ request แรก

9. การรับบน Docker

รันทั้ง stack (front-end + back-end + Postgres) บน Docker ได้เลย โดยไม่ต้องติดตั้งอะไรบนเครื่อง host นอกจาก Docker เอง:

```
docker compose up --build
```

ลำดับการบูตจัดการให้อัตโนมัติ: Postgres เริ่มและ healthy ก่อน → container `migrate` แบบ one-shot รับ `prisma migrate deploy` → แอปเริ่มที่ `http://localhost:3000`

การเข้าถึง LLM จากในคอนเทนเนอร์

- **Ollama บนเครื่อง host (ค่า default):** compose file ชี้ `OLLAMA_BASE_URL` ไปที่ `host.docker.internal:11434` ซึ่งเข้าถึง Ollama instance ที่รันอยู่บนเครื่องเราได้
- **OpenRouter:** สร้างไฟล์ `.env` ไว้ข้าง `docker-compose.yml` พร้อมทั้ง `LLM_PROVIDER=openrouter` และ `OPENROUTER_API_KEY=sk-or-...` — ไม่ต้องใช้ Ollama เลย เขียนไว้ในเอกสาร แต่ยังไม่ได้อัปโหลด

ตัวเลือก OpenRouter ด้านบนถูกเขียนไว้ใน `docker-compose.yml` ก็จริง แต่ไม่มีโค้ดฝั่งแอปพลิเคชันรองรับเลย — ดูหัวข้อ 2 ข้อ 1 อย่าพึ่งพาตัวเลือกนี้

ฐานข้อมูลถูกเก็บไว้ใน volume ชื่อ `db-data` ; คำสั่ง `docker compose down -v` จะล้างข้อมูลทิ้งทั้งหมด

10. Team Workflow

งานถูกแบ่งเป็น 2 สายที่ทำงานคู่ขนานกันได้โดยไม่เหยียบกัน:

สาย 1 — Prompt Testing		สาย 2 — DevOps	
ดูแล	Prompt templates, พฤติกรรม agent, การเลือกโมเดล/provider, คุณภาพผลลัพธ์	Docker, database migrations, ตั้งค่า environment, deployment	
ไฟล์ที่ รับผิดชอบ	<code>src/ai/**</code> , <code>src/lib/presentation/*prompt*</code> , <code>src/lib/model-picker.ts</code> , <code>src/app/api/presentation/**</code>	<code>Dockerfile</code> , <code>docker-compose.yml</code> , <code>prisma/migrations/**</code> , <code>prisma/schema.prisma</code> , <code>.env.example</code> , ตั้งค่า Vercel	
เวิร์กสเต	รันแอปกับ OpenRouter (<code>LLM_PROVIDER=openrouter</code>) เพื่อผลลัพธ์ที่ reproduce ได้ หรือใช้ Ollama ในเครื่องเพื่อนลูปทดสอบฟรี ตัดสินผลลัพธ์ด้วย slide audit agent ยังไม่จริง — ดูหัวข้อ 2	<code>docker compose up --build</code> ต้องบูตแอปที่ใช้งานได้จาก checkout สะอาด ๆ ทุกครั้ง; การเปลี่ยน schema ต้อง ship เป็น SQL migration ผ่าน <code>pnpm db:migrate:dev</code>	
ขอบเขต PR	แค่เฉพาะ prompt/agent เท่านั้น — ห้ามแตะไฟล์ infra	แค่เฉพาะ infra เท่านั้น — ห้ามแตะ prompt	

กติกาที่ต้องยึด:

- การเปลี่ยน schema ถือเป็นงานสาย **DevOps** เสมอ แม้จะมี PR ฝั่ง prompt ที่ต้องใช้มันก็ตาม — ให้แยก PR ออกจากกัน
- การเปลี่ยน prompt ตัดสินด้วยคะแนนจาก verification agent ไม่ใช่ดูผลลัพธ์ที่เจอครั้งเดียวแล้วดูดี
- ทั้งสองสาย branch จาก `main` แล้ว merge ผ่าน PR เท่านั้น ห้าม push ตรงไปที่ `main`

11. วิธีใช้งาน

1. เปิดแอป (`pnpm dev`) แล้วเข้า `http://localhost:3000` — จะเข้าสู่ `/presentation` กันที่ในฐานะ demo admin user
2. สร้างพรินเทชันใหม่จาก `/presentation/create` : อธิบายหัวข้อ ตรวจสอบ/แก้ไข outline ที่ generate มา แล้วค่อยสร้างเดคเต็ม
3. ใน editor ใช้รับสำเร็จรูปหรือสร้างเองได้ แก้สไลด์ได้โดยตรง หรือใช้ chat agent ในตัวเพื่อขอให้แก้ไข (layout, รูปภาพ, สร้าง/regenerate สไลด์, เปลี่ยนธีม)
4. Export เป็น `.pptx` พรินต์ผ่านเบราว์เซอร์โดยตรง หรือสร้างลิงก์แชร์แบบสาธารณะ

12. โครงสร้างโปรเจกต์

แผนผังคร่าว ๆ ของ `src/` :

<code>src/</code>	
<code>├─ app/</code>	Next.js App Router routes
<code>│ ├─ presentation/</code>	ส่วนหลักของแอป: flow การสร้าง, หน้าระหว่าง
<code>│ │</code>	<code>generate</code> , <code>editor/viewer</code> ([id]/)
<code>│ ├─ share/</code>	หน้าแชร์แบบ read-only สาธารณะ
<code>│ └─ api/</code>	Route handlers: chat agent ของพร็เซสเทชั่น
<code>│</code>	(<code>agent/presentation/</code>), <code>generate outline/</code>
<code>│</code>	<code>slide/image/diagram</code> , <code>endpoint auth</code> ที่
<code>│</code>	stub ไว้, route ของ <code>UploadThing</code>
<code>├─ actions/</code>	Server Actions สำหรับ <code>generate รูป</code> ,
<code>│</code>	CRUD ของ <code>notebook/presentation</code> ,
<code>│</code>	เครื่องมือ <code>image-studio</code>
<code>│</code>	
<code>├─ ai/</code>	Agent สำหรับแก้ไขพร็เซสเทชั่น:
<code>│ ├─ agents/presentation/</code>	<code>createAgent.ts</code> – LangGraph agent พร้อม
<code>│ │</code>	chat memory เก็บใน Supabase
<code>│ ├─ tools/</code>	เครื่องมือแก้ไขสไลด์/พิมพ์/รูปภาพ + web search
<code>│ └─ lib/</code>	Postgres checkpointing, middleware
<code>│</code>	สำหรับเนื้อหาที่ paste เข้ามา
<code>│</code>	
<code>├─ components/</code>	
<code>│ ├─ notebook/</code>	ส่วนหลักของ UI outline สไลด์, UI พิมพ์,
<code>│ │</code>	<code>editor plugin</code> , <code>image editor</code> ("Notebook"
<code>│ │</code>	หมายถึงโปรเจกต์พร็เซสเทชั่น 1 อัน ไม่ใช่
<code>│ │</code>	โปรเจกต์จัดโน้ตแยกต่างหาก มี sub-mode
<code>│ │</code>	<code>notes/</code> เล็ก ๆ ที่ยังอยู่ข้างต้นด้วย)
<code>│ └─ presentation/</code>	Chrome ของแอป (<code>sidebar</code> , <code>edit panel</code> ,
<code>│</code>	<code>zoom/scroll</code> , <code>present mode</code>) ที่ประกอบ
<code>│</code>	ขึ้นส่วนจาก <code>components/notebook/</code>
<code>│</code>	
<code>├─ lib/</code>	
<code>│ ├─ model-picker.ts</code>	ตัว resolve LLM แบบ Ollama อย่างเดียว
<code>│ │</code>	(ดูหัวข้อ 3)
<code>│ ├─ notebook/</code>	Data model สำหรับแนบไฟล์ต้นฉบับเข้ากับ
<code>│ │</code>	โปรเจกต์พร็เซสเทชั่น, timeline กิจกรรม agent
<code>│ ├─ presentation/themes.ts</code>	นิยามธีมสำเร็จรูป
<code>│ └─ observability/</code>	Logger แบบ structured ที่เขียนเองและ
<code>│</code>	แสดงผลทาง console อย่างเดียว ไม่มีบริการ
<code>│</code>	ภายนอก (ไม่มี Sentry/PostHog ฯลฯ)
<code>│</code>	
<code>├─ server/</code>	
<code>│ ├─ auth.ts</code>	ตัว stub demo user
<code>│ ├─ ai/</code>	แปลง message ระหว่าง LangChain ↔ AI SDK
<code>│ └─ share/</code>	การตรวจสิทธิ์ของลิงก์แชร์
<code>│</code>	
<code>├─ config/, constants/</code>	Preset ขนาด/รูปแบบสไลด์, แดตาล็อกโมเดล
<code>│</code>	รูปภาพของ FAL, template กราฟ infographic
<code>│</code>	
<code>└─ provider/</code>	React provider ระดับบนสุด (<code>session</code> ,
	<code>React Query</code> , <code>theme</code>)

13. ปัญหาที่รู้อยู่แล้ว

- ระบบค้นหาเอกสาร/RAG ทำโครงไว้แล้วแต่ยังไม่ **implement** — data model ของไฟล์แนบใน notebook มีฟิลด์ `ragId` / `processingStatus` อยู่ และประกาศ `PINECONE_API_KEY` ไว้แล้ว แต่ยังไม่ มี vector-search tool ต่อเข้ากับ agent จริง ๆ อย่างคาดหวังว่าฟีเจอร์ "คุยกับเอกสารที่อัปโหลด" จะใช้งานได้ตอนนี้
- หน้า `/auth/signin` ยังแสดงปุ่ม "**Sign in with Google**" ซึ่งกดไปแล้วไม่ได้ไปไหน — เป็นของหลงเหลือจาก upstream เพราะระบบ auth ถูก stub ไว้ ไม่มีอันตรายแต่ทำให้งงถ้าไปเจอเข้า
- การล็อกฟีเจอร์ "**admin-only**" บางโมเดลรูปภาพไม่มีผลจริงใน **build** นี้ — demo user มี `role: ADMIN` เสมอ ฟีเจอร์เหล่านั้นเลยใช้ได้กับทุกคน
- ตั้งค่าไว้ทั้ง Biome และ Prettier แต่ Biome เป็นตัวหลัก (คือสิ่งที่สคริปต์ `lint` / `check` รัน)

14. การมีส่วนร่วม (Contributing)

เริ่มต้น

1. Fork repository บน GitHub
2. Clone fork ของตัวเอง:

```
git clone git@github.com:Suppakris/Ai102.git
cd Ai102
```

3. ตั้งค่า development environment:

```
pnpm install
```

4. สร้าง branch ใหม่สำหรับสิ่งที่จะ contribute:

```
git checkout -b feature/your-feature-name
```

ขั้นตอนการพัฒนา

1. แก้ไขในเครื่องตัวเอง
2. ทำตาม code style ที่มีอยู่
3. Commit ด้วยข้อความที่ชัดเจน
4. Push branch ไปที่ fork ของตัวเอง
5. เปิด pull request จาก fork ไปที่ repository หลัก

ขั้นตอน Pull Request

1. ตั้งชื่อ PR ให้ชัดเจนและสื่อความหมาย
2. ใส่คำอธิบายที่ละเอียดว่าการแก้ไขนี้ทำอะไร และทำไมควรถูกรวมเข้าไป
3. ลิงก์ issue ที่เกี่ยวข้อง (ถ้ามี)
4. รอ maintainer รีวิว
5. แก้ตาม feedback ที่ได้รับ
6. เมื่อ approve แล้ว maintainer จะ merge ให้

มาตรฐานโค้ด

- ทำตาม code style เดิมที่มีอยู่ในโปรเจกต์
- ตั้งชื่อตัวแปรและฟังก์ชันให้สื่อความหมาย
- เขียนคอมเมนต์ให้ชัดเจนสำหรับ logic ที่ซับซ้อน
- ให้แต่ละฟังก์ชันทำหน้าที่เดียว โฟกัสชัดเจน
- ทำตามแนวทาง TypeScript ที่ดี และรักษา type safety ไว้

แนวทางการ Commit

ใช้รูปแบบ conventional commit:

Prefix	ความหมาย
feat:	ฟีเจอร์ใหม่
fix:	แก้บั๊ก
docs:	แก้ไขเอกสาร
style:	แก้ formatting
refactor:	Refactor โค้ด
chore:	แก้ build process หรือเครื่องมือเสริม

การแจ้งปัญหา (Issue)

1. เช็คว่ามีอยู่แล้วหรือยังในหน้า [Issues](#)
2. ถ้ายังไม่มี ให้สร้าง issue ใหม่โดยใช้ template ที่เหมาะสม
3. ใส่ขั้นตอนการ reproduce บั๊กให้ละเอียด
4. ถ้าเป็น feature request ให้อธิบาย use case และประโยชน์ที่จะได้

15. License

ใช้สัญญาอนุญาต MIT ลิขสิทธิ์เดิมเป็นของทีม ALLWEONE ส่วน fork นี้เป็นงานดัดแปลงที่ทำขึ้นเพื่อใช้ในงานเรียน